Model Details

We for now assume that cell type information is available either through manual gating or some clustering algorithm or both.

Notations

Suppose that we have $t = 1, \cdots, T$ detected transcripts, $c = 1, \cdots, C$ segmented cells, $g = 1, \cdots, G$ genes, and $k = 1, \cdots, K$ cell types.

Associated with each transcript t, we have its currently assigned cell $a_t \in 1, \cdots, C$, its nearest neighbor cell of a different cell type $b_t \in 1, \cdots, C$, its gene type $r_t \in 1, \cdots, G$, and its spatial location $u_t \in \mathbb{R}^2$. We let $\delta_t \in 0, 1$ be an indicator for reassignment, where 0 means remaining the same (assigning to a_t) and 1 means assigning to b_t. We further construct a d-dimensional vector $z_t \in \mathbb{R}^d$ as a series of explanatory variables for computing reassignment probability.

For cell c we let $\theta_c \in 1, \cdots, K$ be its cell type, v_c be a vector describing its location (this can be simply its cell centroid or coordinates of its cell boundary polygon vertices) and $x_c = [x_c^1, \cdots, x_c^G]$ be the observed gene counts (or noisy gene counts). We then let $y_c = [y_c^1, \cdots, y_c^G]$ be the unobserved true gene counts.

Finally, we let $||*||$ denote the cardinality of the enclosed set.

Therefore, if we assume that θ_c are known, we introduced two types of latent RVs that we wish to infer δ_t and y_c. We assume that z_t is completely determined given x_c, v_c and a_t, b_t, r_t, u_t (distance computation and differential analysis).

Model

Although we can model the observed gene counts, to align with our current pipeline, we choose to model the cell types. Given the size of a typical SRT data, we opt for VI. (For other alternatives, see google doc for their sketches).

To get started, we note that given a gene g and a given cell c, we have

$$y_c^g = x_c^g - ||t : \delta_t = 1 \text{ and } a_t = c \text{ and } r_t = g|| + ||t : \delta_t = 1 \text{ and } b_t = c \text{ and } r_t = g||$$

$$\log p(\theta_c | x_c, v_c, a_t, b_t, r_t, u_t) = \iint p(.) \log \frac{p(\theta_c, y_c, \delta_t | x_c, v_c, a_t, b_t, r_t)}{p(y_c, \delta_t | x_c, v_c, \theta_c, a_t, b_t, r_t, u_t)} dy_c d\delta_t,$$

where

$$p(.) = p(y_c, \delta_t | x_c, v_c, \theta_c, a_t, b_t, r_t, u_t)$$

On the numerator within the logarithm, we impose the structure:

$$p_\phi(\theta_c | y_c) p(y_c | x_c, \delta_t, a_t, b_t, r_t, u_t) p(\delta_t)$$

The first term means that the cell type is determined (probabilistically) through the true gene counts. The second term describes the reassignment process. The third term implies our belief that without cell type information, the reassignment is homogenous across all transcripts.

On the denominator within the logarithm, we decompose as follows:

$$p(y_c|x_c,\delta_t,a_t,b_t,r_t,u_t)p(\delta_t|x_c,v_c,\theta_c,a_t,b_t,r_t,u_t)$$

Notice that the first term can be cancelled out. Although we do not know the true posterior (the second term), we can approximate it via

$$q_\beta(\delta_t|x_c,v_c,\theta_c,a_t,b_t,r_t,u_t)$$

As we assume that z_t is determined by the information that we are conditioning on, we can simply re-write it as

$$q_\beta(\delta_t|z_t)$$

Therefore, the ELBO is

$$\log p(\theta_c|x_c,v_c,a_t,b_t,r_t,u_t) \geq \iint q_\beta(.)\log\frac{p_\phi(\theta_c|y_c)p(\delta_t)}{q_\beta(\delta_t|z_t)}dy_c d\delta_t$$

We assume that $p_\phi(\theta_c|y_c)=\prod_{c=1}^{C}p_\phi(\theta_c|y_c)$, $p(\delta_t)=\prod_{t=1}^{T}p(\delta_t)$, and $q_\beta(\delta_t|z_t)=\prod_{t=1}^{T}q_\beta(\delta_t|z_t)$, where $p(\delta_t=0)=0.5$, $q_\beta(\delta_t|z_t)=\text{Bernoulli}(\frac{1}{1+\exp(-z_t^T\beta)})$, and $p_\phi(\theta_c|y_c)=\text{Category}(\text{softmax}(y_c^T\phi))$. We use the Gumbel softmax trick to sample Bernoulli RVs and we can learn all the parameters via gradient decent.

The loss is then $\text{cross entropy loss}(\theta_c,\text{logits})+\text{KL Divergence}$

Sampling

We will patchify the data. Within each patch, we make sure that there are enough cells (but not too many).

z_t

Currently, z_t is made up of three components: distance, results from one-vs-one differential analysis, and results from one-vs-rest differential analysis.

Distance is computed as

$$\log_2(\frac{\text{distance to self centroid}}{\text{distance to neighbor centroid}})$$

For one-vs-one differential, we collect the log2FoldChange (lFC) and the adjusted p-value (padj) (transformed via $-\log_{10}()$). The feature is constructed as $\text{lFC}*\text{padj}$.

Quick summary

The framework allows simultaneous learning of all the parameters without grid search. However, there are several issues

Issue 1

Features. We need to contemplate on the features we construct.

Issue 2

Based on the ELBO alone, nothing is telling the model that reassigning a transcript that's far away from the boundary is bad. Three possible solutions: 1. construct better features; 2. add penalty on distances; 3. make the coefficients are positive. (I have observed negative or close to 0 coefficient corresponding to distance feature).

Issue 3

Cell typing. We are assuming cell typing is fixed. Do we need to relax this assumption? If so, we will need to consider modeling the observed gene counts x_c.

Issue 4

Patch generation. Patch size.

Issue 5

Previously we are using entropy. We can still add entropy into the loss. Or we can use entropy as a validation. A potential drawback of entropy is that its minimum is attained at various points. When the classifier is super certain that the cell is or is not of a type, the entropy will decrease in both cases.

Issue 5

We kind of justify why we do not need to re-do cell typing if we model the observed gene counts.

Now, we only observe a_t, u_t, r_t, v_c, x_c . Note that we do not have b_t since it's determined by θ_c.

If we model

$$p(x_c|v_c, a_t, u_t, r_t)$$

following a similar step, we have on the numerator in the logarithm

$$p(x_c, \delta_t, \theta_c, y_c|v_c, a_t, u_t, r_t)$$

we decompose it into the product of

$$p(x_c|y_c, \theta_c, \delta_t, v_c, a_t, u_t, r_t),$$

which is deterministic since given θ_c, we can now figure out b_t, and

$$p(y_c|\theta_c, \delta_t, v_c, a_t, u_t, r_t) = p(y_c|\theta_c),$$

which could be a fancy zero-inflated negative binomial model or whatever, and

$$p(\theta_c)p(\delta_t),$$

assuming that we do not wish to model impact of spatial information on cell typing.

On the denominator in the logarithm, we have

$$q(\delta_t, \theta_c, y_c|x_c, v_c, a_t, u_t, r_t)$$

we decompose it into the product of

$$q(y_c|x_c, v_c, \delta_t, \theta_c, a_t, u_t, r_t),$$

which is deterministic since given θ_c, we can now figure out b_t, and

$$q(\delta_t|x_c, v_c, \theta_c, a_t, u_t, r_t),$$

which would be the same as the one we have in the current model as we can know compute z_t, and

$$q(\theta_c|x_c, v_c, a_t, u_t, r_t)$$

Of course we model this however we want. But notice that the condition information only contains the observed gene counts. As nothing is updated during the learning, we do not need to re-do cell typing under this setting.

A con of modeling gene counts versus cell type is that it involves more parameters (2-3 times more).

Now, back to our original setting where we model the cell type instead of the gene counts. Maybe we can justify as follows:

$$p(\theta_c|x_c, v_c, a_t, b_t, u_t, r_t) = \frac{p(x_c|\theta_c, v_c, a_t, b_t, u_t, r_t)p(\theta_c|v_c, a_t, b_t, u_t, r_t)}{p(x_c|v_c, a_t, b_t, u_t, r_t)}$$

If we choose not to incorporate spatial information into cell typing, we can treat the second term on the numerator as fixed. The denominator is what we just discussed if we remove b_t. Therefore, once the initial cell typing is done, we do not need more iterations.